

Ответы к экзамену по NLP

Содержание

0.1	Вариант 10	2
0.1.1	1. Теоретический вопрос	2
0.2	2. Вопросы с открытым ответом	3
0.3	3. Практико-ориентированное задание	3
0.3.1	Формула scaled dot-product attention	5
0.3.2	Что замораживать при маленьком корпусе	5
0.4	Вариант 27	5
0.4.1	1. Теоретический вопрос	5
0.5	2. Вопросы с открытым ответом	5
0.6	3. Практико-ориентированное задание	6
0.6.1	Формула усечённого SVD-разложения	7
0.7	Вариант 22	7
0.7.1	1. Теоретический вопрос	7
0.8	2. Вопросы с открытым ответом	7
0.9	3. Практико-ориентированное задание	8
0.9.1	Формула greedy decoding	10
0.9.2	Почему примеры в prompt помогают	10
0.10	Вариант 21	10
0.10.1	1. Теоретический вопрос	10
0.11	2. Вопросы с открытым ответом	10
0.12	3. Практико-ориентированное задание	11
0.12.1	Формула MLE-оценки вероятности биграммы	12
0.12.2	Cross-entropy и perplexity	12
0.12.3	Что будет без сглаживания	12
0.13	Вариант 30	12
0.13.1	1. Теоретический вопрос	12
0.14	2. Вопросы с открытым ответом	13
0.15	3. Практико-ориентированное задание	14
0.15.1	Формула матрицы совместной встречаемости	14
0.16	Вариант 28	15
0.16.1	1. Теоретический вопрос	15
0.17	2. Вопросы с открытым ответом	15
0.18	3. Практико-ориентированное задание	16
0.18.1	Формула итогового представления токена в BiRNN	17
0.18.2	Почему GRU быстрее LSTM	17
0.19	Вариант 17	17
0.19.1	1. Теоретический вопрос	17
0.20	2. Вопросы с открытым ответом	17
0.21	3. Практико-ориентированное задание	18
0.21.1	Формула negative sampling	20
0.21.2	Вероятность выбора отрицательного примера	20
0.21.3	Что будет с редкими словами при случайных отрицательных примерах	20
0.22	Вариант 18	21
0.22.1	1. Теоретический вопрос	21
0.23	2. Вопросы с открытым ответом	21

0.243. Практико-ориентированное задание	22
0.24.1 Формула обновления долговременной памяти LSTM	23
0.25 Вариант 7	23
0.25.11. Теоретический вопрос	23
0.262. Вопросы с открытым ответом	23
0.273. Практико-ориентированное задание	24
0.27.1 Формула матрицы совместной встречаемости	25
0.28 Вариант 29	26
0.28.11. Теоретический вопрос	26
0.292. Вопросы с открытым ответом	26
0.303. Практико-ориентированное задание	27
0.30.1 Формула обучения центрального слова по контексту	28
0.31 Вариант 20	28
0.31.11. Теоретический вопрос	28
0.322. Вопросы с открытым ответом	29
0.333. Практико-ориентированное задание	30
0.33.1 Формула усечённого SVD-разложения	30
0.34 Вариант 40	30
0.34.11. Теоретический вопрос	30
0.352. Вопросы с открытым ответом	31
0.363. Практико-ориентированное задание	32
0.36.1 Формула матрицы совместной встречаемости	33
0.37 Вариант 6	33
0.37.11. Теоретический вопрос	33
0.382. Вопросы с открытым ответом	33
0.393. Практико-ориентированное задание	34
0.39.1 Формула распределения вероятностей по интентам	36
0.39.2 Почему сущности извлекают как метки токенов	36
0.40 Вариант 25	36
0.40.11. Теоретический вопрос	36
0.412. Вопросы с открытым ответом	36
0.423. Практико-ориентированное задание	37
0.42.1 MLE-оценка вероятности биграммы	38
0.42.2 Связь cross-entropy и perplexity	38
0.43 Вариант 3	38
0.43.11. Теоретический вопрос	38
0.442. Вопросы с открытым ответом	39
0.453. Практико-ориентированное задание	39
0.45.1 MLE-оценка вероятности биграммы	40
0.45.2 Связь cross-entropy и perplexity	41
0.46 Вариант 27	41
0.46.11. Теоретический вопрос	41
0.472. Вопросы с открытым ответом	41
0.483. Практико-ориентированное задание	42
0.48.1 Формула аддитивного внимания	43
0.48.2 Какую проблему решает attention	43

0.1 **Вариант 10**

0.1.1 **1. Теоретический вопрос**

Не отвечаю по инструкции.

0.2 2. Вопросы с открытым ответом

1. Формула закона Ципфа и что такое ранг. Закон Ципфа:

$$f(w) \approx \frac{C}{r(w)^\alpha}, \quad \alpha \approx 1$$

где $f(w)$ — частота слова, $r(w)$ — ранг слова в списке по убыванию частоты, C — константа. Ранг 1 имеет самое частое слово, ранг 2 — второе по частоте и т.д.

2. Два применения закона Ципфа в NLP. Во-первых, он помогает находить стоп-слова: самые частые слова часто малоинформативны. Во-вторых, он помогает анализировать словарь корпуса: редкие слова можно отфильтровывать или обрабатывать отдельно.

3. Допущение Маркова n-го порядка. Считается, что вероятность текущего слова зависит не от всей истории, а только от последних (n) слов:

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-n}, \dots, w_{t-1})$$

Это упрощает языковую модель и лежит в основе n-грамм.

4. nn.RNN, batch_first=True, hidden_size=N, вход (B, T, D). Для однослойной обычной RNN:

$$output : (B, T, H), \quad h_n : (1, B, H)$$

output содержит скрытые состояния для всех токенов последовательности, а h_n — последнее скрытое состояние.

5. Однослойный двунаправленный nn.LSTM, вход (B, T, D). Форма output:

$$(B, T, 2H)$$

Потому что для каждого токена объединяются два состояния: прямое направление слева направо и обратное направление справа налево.

6. Интент в чат-боте. Интент — это намерение пользователя, то есть цель его сообщения. Например: «Какая завтра погода?» → weather_request; «Закажи пиццу» → food_order; «Хочу отменить бронь» → cancel_booking.

7. Что замораживается и что обучается в LoRA. В LoRA исходные веса модели (W) замораживаются, а обучаются только маленькие матрицы низкого ранга (A) и (B):

$$W' = W + \Delta W, \quad \Delta W = AB$$

Так модель дообучается почти как при fine-tuning, но обучаемых параметров намного меньше.

8. Инициализация матриц A и B в LoRA. Обычно (A) инициализируется маленькими случайными значениями, а (B) — нулями. Тогда в начале $\Delta W = AB = 0$, и модель сначала ведёт себя как исходная предобученная модель.

9. Для чего применяется квантизация весов. Квантизация нужна, чтобы уменьшить размер модели, расход памяти и ускорить инференс. Методы: fp32 → fp16/bfloat16, int8/int4-квантизация, симметричная и асимметричная квантизация, линейная и range-based квантизация.

10. Преимущества vLLM перед базовым Hugging Face inference. vLLM быстрее обслуживает запросы за счёт эффективного управления KV-cache через PagedAttention. Также он поддерживает динамический батчинг, лучше использует GPU и даёт меньшую задержку при большом числе запросов.

0.3 3. Практико-ориентированное задание

```

from datasets import Dataset
from transformers import (
    BertTokenizerFast,
    BertForSequenceClassification,
    TrainingArguments,
    Trainer
)

# 1. Данные: короткие отзывы и бинарные метки
texts = [
    "Фильм отличный, мне понравилось",
    "Очень плохой товар",
    "Сервис хороший",
    "Ужасное качество"
]
labels = [1, 0, 1, 0] # 1 – positive, 0 – negative

dataset = Dataset.from_dict({"text": texts, "label": labels})
dataset = dataset.train_test_split(test_size=0.25)

# 2. Загружаем предобученный BERT и токенизатор
model_name = "bert-base-multilingual-cased"
tokenizer = BertTokenizerFast.from_pretrained(model_name)

def tokenize(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

dataset = dataset.map(tokenize, batched=True)
dataset = dataset.remove_columns(["text"])
dataset.set_format("torch")

# 3. Загружаем модель и заменяем классификационную голову
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=2)
model.classifier = nn.Linear(model.config.hidden_size, 2)

# 4. Для маленького корпуса заморозим BERT и обучим голову + последний слой
for p in model.bert.parameters():
    p.requires_grad = False

for p in model.bert.encoder.layer[-1].parameters():
    p.requires_grad = True

for p in model.classifier.parameters():
    p.requires_grad = True

# 5. Дообучение
args = TrainingArguments(
    output_dir="./bert_sentiment",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    learning_rate=2e-5,
    logging_steps=1,

```

```

    report_to="none"
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"]
)

trainer.train()

```

0.3.1 Формула scaled dot-product attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

где (Q) — матрица запросов, (K) — матрица ключей, (V) — матрица значений, (d_k) — размерность ключей. Произведение (QK^T) показывает похожесть запросов и ключей, деление на $\sqrt{d_k}$ стабилизирует значения, softmax превращает их в веса внимания, а умножение на (V) даёт итоговые контекстные представления.

0.3.2 Что замораживать при маленьком корпусе

На маленьком размеченном корпусе обычно замораживают большую часть BERT, потому что иначе модель быстро переобучится. Обучают новую классификационную голову и иногда последние 1-2 слоя BERT. Выбор зависит от размера данных, отличия новой задачи от предобучения и доступных вычислительных ресурсов.

0.4 Вариант 27

0.4.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.5 2. Вопросы с открытым ответом

1. Зачем в обучении word2vec применяется negative sampling и что он заменяет?

Negative sampling нужен, чтобы не считать softmax по всему словарю, потому что словарь может быть очень большим. Вместо этого модель учится отличать правильную пару «слово-контекст» от нескольких случайных отрицательных примеров:

$$\log \sigma(v_c^T v_w) + \sum_{i=1}^k \log \sigma(-v_{n_i}^T v_w)$$

где (v_w) — вектор слова, (v_c) — вектор правильного контекста, (v_{n_i}) — отрицательные слова.

2. Минимум два преимущества fastText перед word2vec. fastText представляет слово как сумму эмбедингов символьных n-грамм, поэтому лучше работает с редкими словами, опечатками и словами вне словаря. Также он лучше подходит для языков с богатой морфологией, например русского, потому что учитывает части слова.

3. Чем обычно инициализируется скрытое состояние RNN на первом шаге? Обычно начальное скрытое состояние задаётся нулевым вектором:

$$h_0 = 0$$

Так делают, потому что на первом шаге у RNN ещё нет предыдущего контекста, от которого можно было бы отталкиваться.

4. Как соотносятся веса ячейки RNN на разных временных шагах? Веса RNN на всех временных шагах одинаковые, то есть используются одни и те же матрицы (W_x) и (W_h). Например:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$
$$h_{t+1} = f(W_x x_{t+1} + W_h h_t + b)$$

Это значит, что одна и та же ячейка применяется к каждому элементу последовательности.

5. Какая функция активации обычно используется для нового скрытого состояния в базовой RNN? В классической RNN обычно используется гиперболический тангенс:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

$\tanh$ сжимает значения в диапазон $(-1;1)$ и помогает получить новое скрытое состояние.

6. Какова роль матрицы Value (V) в формуле Attention(Q,K,V)? Формула внимания:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

(Q) и (K) определяют, на что обратить внимание, а (V) содержит сами значения, которые смешиваются по найденным весам внимания. То есть (V) — это информация, которую модель реально извлекает.

7. Чем cross-attention отличается от self-attention? В self-attention (Q), (K), (V) берутся из одной и той же последовательности. В cross-attention запросы (Q) берутся из одной последовательности, а ключи (K) и значения (V) — из другой, например в переводе decoder смотрит на выходы encoder. В лекции cross-attention описан как работа с двумя разными последовательностями.

8. Почему Transformer требует positional encoding? Self-attention сам по себе не знает порядок слов: для него слова просто набор токенов, между которыми считаются связи. Positional encoding добавляет информацию о позиции токена, чтобы модель отличала «кошка поймала мышь» от «мышь поймала кошку».

9. Дайте определение slot filling в диалоговой системе. Slot filling — это извлечение конкретных параметров из реплики пользователя для выполнения интента. Например: «Забронируй столик в ресторане Утка завтра на двоих» → restaurant=Утка, date=завтра, people=2.

10. Какие этапы проходят документы в базовом RAG-пайплайне перед добавлением в промпт? Документы сначала очищаются и разбиваются на фрагменты/chunks, затем для них строятся эмбединги и они сохраняются в векторный индекс. После запроса пользователя находятся top-k наиболее близких фрагментов, они могут быть отфильтрованы или rerank-нуты, а затем добавляются в промпт как контекст.

0.6 3. Практико-ориентированное задание

```
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import normalize
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

```
# Корпус новостных документов
```

```
docs = [
```

```
    "Правительство обсудило новые меры поддержки экономики",
```

```
    "Футбольная команда выиграла важный матч чемпионата",
```

```

    "Центробанк сообщил об изменении ключевой ставки",
    "Ученые разработали новую модель искусственного интеллекта",
    "Спортсмен установил новый мировой рекорд"
]

# 1. Матрица документ-термин с TF-IDF весами
vectorizer = TfidfVectorizer()
X_dt = vectorizer.fit_transform(docs)    # shape: документы x термины

# Если нужна именно терм-документная матрица:
X_td = X_dt.T                          # shape: термины x документы

# 2. Усеченное SVD-разложение
k = 2
svd = TruncatedSVD(n_components=k, random_state=42)
D_k = svd.fit_transform(X_dt)          # документы в k-мерном пространстве
D_k = normalize(D_k)

# 3. Проекция нового запроса и выдача top-k
query = "новости экономики и ключевая ставка"
q = vectorizer.transform([query])
q_k = normalize(svd.transform(q))

top_k = 3
scores = cosine_similarity(q_k, D_k).ravel()
best_ids = np.argsort(scores)[::-1][:top_k]

for i in best_ids:
    print(f"score={scores[i]:.3f} | {docs[i]}")

```

0.6.1 Формула усечённого SVD-разложения

$$X \approx U_k \Sigma_k V_k^T$$

где (X) — TF-IDF матрица терм-документ, (U_k) — матрица терминов в скрытом (k)-мерном пространстве, Σ_k — диагональная матрица (k) главных сингулярных значений, (V_k^T) — представления документов в этом пространстве.

Усечённое SVD лучше полного для разреженной TF-IDF матрицы, потому что оставляет только главные скрытые темы и отбрасывает шум. Оно быстрее, требует меньше памяти и удобнее для поиска похожих документов.

0.7 Вариант 22

0.7.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.8 2. Вопросы с открытым ответом

1. Зачем в word2vec применяется negative sampling и что он заменяет? Negative sampling заменяет полный softmax по всему словарю, который слишком дорогой при большом числе слов. Модель учится отличать реальную пару «слово-контекст» от нескольких случайных отрицательных примеров:

$$\log \sigma(v_c^T v_w) + \sum_{i=1}^k \log \sigma(-v_{n_i}^T v_w)$$

где (v_w) — вектор слова, (v_c) — вектор правильного контекста, $(v_{\{n_i\}})$ — отрицательные примеры.

2. Два преимущества fastText перед word2vec. fastText учитывает не только слово целиком, но и символьные n-граммы, то есть части слова. Поэтому он лучше работает с редкими словами, опечатками и словами вне словаря, особенно в языках с богатой морфологией.

3. Чем обычно инициализируется скрытое состояние RNN на первом шаге? Обычно начальное состояние задают нулевым вектором:

$$h_0 = 0$$

Так делают, потому что на первом шаге у модели ещё нет предыдущего контекста.

4. Как соотносятся веса RNN на разных временных шагах? В RNN на всех временных шагах используются одни и те же веса. Например:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

$$h_{t+1} = \tanh(W_x x_{t+1} + W_h h_t + b)$$

Это даёт возможность обрабатывать последовательности разной длины одной и той же ячейкой.

5. Какая функция активации используется в базовой RNN? В базовой RNN обычно используется $\tanh$:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Она сжимает значения в диапазон $(-1; 1]$ и формирует новое скрытое состояние.

6. nn.GRU, batch_first=True, hidden_size=N, вход (B, T, D). Формы output и h_n. Для однослойной однонаправленной GRU:

$$output : (B, T, H), \quad h_n : (1, B, H)$$

output содержит скрытые состояния для всех токенов, а h_n — последнее скрытое состояние слоя.

7. Преимущества Transformer перед RNN. Transformer можно обучать параллельно, потому что он не обязан идти по токенам строго последовательно. Также self-attention лучше работает с дальними зависимостями, потому что любой токен может напрямую учитывать любой другой токен.

8. Основные блоки Transformer и их роль. Главные блоки: self-attention, feed-forward network, residual connection, layer normalization и positional encoding. Self-attention ищет связи между токенами, FFN преобразует признаки, residual и normalization стабилизируют обучение, positional encoding добавляет порядок слов.

9. Формула residual connection и какую проблему она решает. Residual connection:

$$y = F(x) + x$$

где (x) — вход слоя, $(F(x))$ — преобразование слоя. Такой пропускной путь помогает бороться с затуханием градиента и упрощает обучение глубоких сетей.

10. Интент в чат-боте. Интент — это намерение пользователя, то есть цель его сообщения. Например: «Какая завтра погода?» → weather_request; «Закажи пиццу» → food_order; «Хочу отменить бронь» → cancel_booking.

0.9 3. Практико-ориентированное задание

```
from transformers import AutoTokenizer, AutoModelForCausalLM, GenerationConfig
```

1. Несколько эталонных задач с подробными решениями


```

examples = [
    {
        "task": "У Пети было 5 яблок, он купил ещё 3. Сколько яблок стало?",
        "solution": "Было 5. Купил ещё 3.  $5 + 3 = 8$ . Ответ: 8."
    },
    {
        "task": "В коробке 12 конфет. 4 конфеты съели. Сколько осталось?",
        "solution": "Было 12. Съели 4.  $12 - 4 = 8$ . Ответ: 8."
    },
    {
        "task": "У Маши 3 пакета по 6 карандашей. Сколько всего карандашей?",
        "solution": "Есть 3 пакета, в каждом по 6.  $3 * 6 = 18$ . Ответ: 18."
    }
]

new_task = "В классе 24 ученика. Их разделили поровну на 4 группы. Сколько учеников в каждой группе?"

# 2. Системный prompt с примерами
system_prompt = """
Ты решаешь школьные арифметические задачи.
Решай пошагово: выпиши данные, выбери действие, посчитай и дай краткий ответ.

Примеры:
"""

for ex in examples:
    system_prompt += f"\nЗадача: {ex['task']}\nРешение: {ex['solution']}\n"

prompt = system_prompt + f"\nТеперь реши новую задачу:\nЗадача: {new_task}\nРешение:"

# 3. Загрузка готовой LLM и настройка генерации
model_name = "Qwen/Qwen2.5-0.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

generation_config = GenerationConfig(
    max_new_tokens=150,
    do_sample=False,      # greedy decoding
    temperature=1.0
)

# 4. Вызов модели и вывод результата
inputs = tokenizer(prompt, return_tensors="pt")

with torch.no_grad():
    output = model.generate(
        **inputs,
        generation_config=generation_config
    )

answer = tokenizer.decode(output[0], skip_special_tokens=True)
print(answer)

```

0.9.1 Формула greedy decoding

$$p(x_t | x_{<t}) = \text{softmax}(z_t)$$

$$x_t = \arg \max_{v \in V} p(v | x_{<t})$$

где (z_t) — логиты модели для следующего токена, (V) — словарь токенов, $(x_{<t})$ — уже сгенерированный контекст. При greedy decoding модель каждый раз выбирает токен с максимальной вероятностью.

0.9.2 Почему примеры в prompt помогают

Добавление решённых примеров показывает модели нужный формат рассуждения: сначала данные, потом действие, потом вычисление и ответ. Это называется in-context learning: веса модели не меняются, но модель по контексту понимает шаблон решения и чаще правильно решает похожие арифметические задачи.

0.10 Вариант 21

0.10.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.11 2. Вопросы с открытым ответом

1. Назовите минимум два преимущества косинусной меры перед евклидовым расстоянием при сравнении документов. Косинусная мера сравнивает направление векторов, а не их длину, поэтому меньше зависит от размера документа. Она хорошо подходит для разреженных TF-IDF-векторов и показывает сходство по относительному распределению слов.

2. nn.RNN, batch_first=True, hidden_size=N, вход (B, T, D). Формы output и h_n. Для однослойной однонаправленной RNN:

$$\text{output} : (B, T, H), \quad h_n : (1, B, H)$$

output содержит скрытые состояния на всех шагах, а h_n — последнее скрытое состояние.

3. Однослойный двунаправленный nn.LSTM, вход (B, T, D). Форма output. Форма выхода:

$$\text{output} : (B, T, 2H)$$

Потому что для каждого токена объединяются два направления: прямое и обратное.

4. Почему Transformer требует positional encoding? Self-attention сам по себе не учитывает порядок слов: он видит токены как набор элементов. Positional encoding добавляет информацию о позиции, чтобы модель различала, например, «кошка поймала мышь» и «мышь поймала кошку».

5. Минимум два преимущества синусоидального positional encoding перед обучаемым. Синусоидальное кодирование не требует дополнительных обучаемых параметров. Также оно может обобщаться на последовательности большей длины, чем были при обучении.

6. Основные слои блока энкодера оригинального Transformer. Блок энкодера состоит из multi-head self-attention, residual connection, layer normalization и position-wise feed-forward network. Self-attention ищет связи между токенами, а feed-forward слой дополнительно преобразует признаки каждого токена.

7. Slot filling в диалоговой системе. Slot filling — это извлечение параметров из реплики пользователя для выполнения интента. Например: «Забронируй столик завтра на двоих» → date=завтра, people=2.

8. LoRA: что замораживается и что обучается? В LoRA исходная матрица весов (W) замораживается, а обучаются только маленькие матрицы (A) и (B):

$$W' = W + \Delta W, \quad \Delta W = AB$$

Так дообучение становится дешёвым по памяти и числу параметров.

9. Как инициализируются матрицы A и B в LoRA? Почему так? Обычно (A) инициализируется маленькими случайными значениями, а (B) — нулями. Тогда в начале $\Delta W = AB = 0$, и модель стартует с поведения исходной предобученной модели.

10. Для чего применяется квантизация весов? Методы. Квантизация нужна, чтобы уменьшить размер модели, расход памяти и ускорить инференс. Методы: fp32 $\rightarrow$ fp16/bfloat16, int8/int4, симметричная и асимметричная квантизация.

0.12 3. Практико-ориентированное задание

```
import math

train = [
    "я люблю машинное обучение",
    "я люблю обработку текста",
    "машинное обучение это интересно"
]

test = [
    "я люблю обучение",
    "обработка текста это интересно"
]

def tokenize(sent):
    return ["<s>"] + sent.lower().split() + ["</s>"]

# 1. Сбор биграмм и униграмм
unigrams = Counter()
bigrams = Counter()

for sent in train:
    tokens = tokenize(sent)
    unigrams.update(tokens[:-1])
    bigrams.update(zip(tokens[:-1], tokens[1:]))

vocab = set()
for sent in train:
    vocab.update(tokenize(sent))
V = len(vocab)

# 2. Сглаженная вероятность биграммы: Laplace smoothing
def prob(w_prev, w):
    return (bigrams[(w_prev, w)] + 1) / (unigrams[w_prev] + V)

# 3. Перплексия на тестовом корпусе
log_prob = 0
N = 0

for sent in test:
    tokens = tokenize(sent)
```

```

for w_prev, w in zip(tokens[:-1], tokens[1:]):
    p = prob(w_prev, w)
    log_prob += math.log(p)
    N += 1

cross_entropy = -log_prob / N
perplexity = math.exp(cross_entropy)

print("Cross-entropy:", cross_entropy)
print("Perplexity:", perplexity)

```

0.12.1 Формула MLE-оценки вероятности биграммы

$$P_{MLE}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

где $C(w_{i-1}, w_i)$ — количество биграмм, а $C(w_{i-1})$ — количество появлений первого слова как контекста.

0.12.2 Cross-entropy и perplexity

$$H = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{i-1})$$

$$PPL = e^H$$

Если используются логарифмы по основанию 2, тогда:

$$PPL = 2^H$$

Перплексия показывает, насколько модель «удивляется» тестовому тексту: чем меньше значение, тем лучше модель предсказывает следующие слова.

0.12.3 Что будет без сглаживания

Если в тесте встретится биграмма, которой не было в обучении, то:

$$P(w_i | w_{i-1}) = 0$$

Тогда $\log 0$ не определён, cross-entropy становится бесконечной, а perplexity тоже становится бесконечной. Поэтому для n-граммных моделей почти всегда применяют сглаживание.

0.13 Вариант 30

0.13.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.14 2. Вопросы с открытым ответом

1. Как соотносятся веса ячейки RNN на разных временных шагах? В RNN на всех временных шагах используются одни и те же веса, то есть параметры ячейки разделяются по времени. Это позволяет одной модели обрабатывать последовательности разной длины.

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$
$$h_{t+1} = \tanh(W_x x_{t+1} + W_h h_t + b)$$

2. Какая функция активации используется для нового скрытого состояния в базовой RNN? Обычно используется гиперболический тангенс $\tanh$:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Она сжимает значения в диапазон $([-1;1])$ и формирует новое скрытое состояние.

3. Что такое эффект бутылочного горлышка в seq2seq на базе RNN? В классическом seq2seq весь смысл входной последовательности сжимается в один вектор последнего скрытого состояния encoder. Из-за этого модель может терять информацию, особенно в длинных предложениях, и плохо передавать дальние зависимости.

4. Роль матрицы Value (V) в Attention. Формула внимания:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

(Q) и (K) определяют веса внимания, а (V) содержит сами значения, которые смешиваются по этим весам.

5. Чем cross-attention отличается от self-attention? В self-attention (Q), (K), (V) берутся из одной последовательности, например слова одного предложения смотрят друг на друга. В cross-attention (Q) берётся из decoder, а (K,V) — из encoder, например при машинном переводе decoder обращается к закодированному исходному тексту.

6. Сущности в запросе: «Забронируйте столик в ресторане Утка на завтра на двоих». Сущности: ресторан = Утка, дата = завтра, количество_людей = двое. Интент здесь — бронирование столика.

7. Что означает SFT в обучении LLM? SFT — это Supervised Fine-Tuning, то есть дообучение модели с учителем. Оно проводится на размеченных данных вида «инструкция/запрос → правильный ответ», часто на диалогах и примерах хороших ответов.

8. Особенность in-context learning по сравнению с fine-tuning. In-context learning не меняет веса модели: примеры задачи просто добавляются в prompt. Плюс — быстро и не требует обучения; минусы — зависит от качества prompt, ограничен длиной контекста и может быть менее стабильным.

9. Сколько параметров добавит LoRA для слоя 1024×1024 , если $(r=8)$? LoRA добавляет две матрицы:

$$A \in \mathbb{R}^{1024 \times 8}, \quad B \in \mathbb{R}^{8 \times 1024}$$

Количество параметров:

$$1024 \cdot 8 + 8 \cdot 1024 = 8192 + 8192 = 16384$$

Итого: **16 384 обучаемых параметра.**

10. Этапы документов в базовом RAG-пайплайне перед добавлением в prompt. Документы очищаются, разбиваются на chunks, затем для фрагментов строятся embeddings и сохраняются в векторную базу. После запроса находятся top-k близких фрагментов, при необходимости ранжируются, и только потом добавляются в prompt как контекст.

0.15 3. Практико-ориентированное задание

```
import numpy as np
import pandas as pd
from collections import Counter

docs = [
    "Кошка поймала мышь",
    "Собака увидела кошку",
    "Кошка и собака живут дома"
]

stop_words = {"и", "в", "на", "а", "но"}

def tokenize(text):
    text = text.lower()
    text = re.sub(r"^[^а-яёа-з ]", " ", text)
    return [w for w in text.split() if w not in stop_words]

# 1. Предобработка корпуса и словарь
corpus = [tokenize(doc) for doc in docs]

vocab = sorted(set(word for doc in corpus for word in doc))
word2id = {word: i for i, word in enumerate(vocab)}

# 2. Инициализация матрицы совместной встречаемости
w = 2
C = np.zeros((len(vocab), len(vocab)), dtype=int)

# 3. Проход по корпусу с окном ±w и заполнение частот
for doc in corpus:
    for i, center_word in enumerate(doc):
        center_id = word2id[center_word]

        left = max(0, i - w)
        right = min(len(doc), i + w + 1)

        for j in range(left, right):
            if i == j:
                continue

            context_word = doc[j]
            context_id = word2id[context_word]
            C[center_id, context_id] += 1

cooc_df = pd.DataFrame(C, index=vocab, columns=vocab)
print(cooc_df)
```

0.15.1 Формула матрицы совместной встречаемости

$$C_{ij} = \text{count}(w_i, w_j)$$

где (C_{ij}) — сколько раз слово (w_j) встретилось в контекстном окне рядом со словом (w_i) . Окно размера (w) означает, что для каждого слова берём до (w) слов слева и до (w) слов справа.

Стоп-слова и слишком частые токены лучше удалять, потому что они часто встречаются почти со всеми словами и засоряют матрицу. Из-за них модель хуже видит полезные смысловые связи между словами.

0.16 Вариант 28

0.16.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.17 2. Вопросы с открытым ответом

1. Минимум два преимущества softmax перед max в глубоком обучении. max выбирает только один самый большой элемент и обнуляет вклад остальных, поэтому через него плохо проходит обучение. softmax превращает числа в вероятности:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Он дифференцируемый и показывает не только лучший класс, но и уверенность модели.

2. Зачем в word2vec применяется negative sampling и что он заменяет? Negative sampling заменяет дорогой softmax по всему словарю. Вместо перебора всех слов модель учится отличать правильную пару «слово-контекст» от нескольких случайных отрицательных слов:

$$\log \sigma(v_c^T v_w) + \sum_{i=1}^k \log \sigma(-v_{n_i}^T v_w)$$

где (v_w) — вектор слова, (v_c) — правильный контекст, (v_{n_i}) — отрицательные примеры.

3. Почему Transformer требует positional encoding? Self-attention сам по себе не знает порядок токенов: без позиций для него последовательность похожа на набор слов. Positional encoding добавляет информацию о месте слова в предложении, чтобы модель различала, например, «кошка поймала мышь» и «мышь поймала кошку».

4. Минимум два преимущества синусоидального positional encoding перед обучаемым. Синусоидальное positional encoding не добавляет обучаемых параметров. Также оно лучше обобщается на длины последовательностей, которых не было при обучении, потому что позиции задаются формулой, а не таблицей.

5. Основные слои блока энкодера оригинального Transformer. Блок энкодера включает multi-head self-attention, residual connection, layer normalization и feed-forward network. Self-attention ищет связи между токенами, FFN преобразует признаки каждого токена, residual connection и normalization стабилизируют обучение.

6. Сущности в запросе «Забронируйте столик в ресторане Утка на завтра на двоих». Интент: бронирование_столика. Сущности: ресторан = Утка, тип — название ресторана; дата = завтра, тип — дата; количество_людей = двое, тип — число гостей.

7. Особенность in-context learning по сравнению с fine-tuning. При in-context learning веса модели не изменяются: примеры задачи добавляются прямо в prompt. Плюсы: не нужно дообучать модель, быстро проверять новые задачи. Минусы: зависит от качества prompt, ограничен длиной контекста и обычно менее стабилен, чем fine-tuning.

8. На основе чего обучается Reward Model в RLHF? Reward Model обучается на человеческих предпочтениях: человеку показывают несколько ответов модели на один запрос, и он выбирает лучший. Пример данных:

```
chosen: "RNN — сеть для последовательностей, которая хранит скрытое состояние."  
rejected: "RNN — это просто обычная нейросеть."
```

Модель учится давать больший reward хорошему ответу.

9. Зачем в RLHF через PPO используется копия исходной base model? Копия исходной модели нужна как reference model, чтобы новая политика не слишком далеко уходила от исходного поведения. Обычно добавляют KL-штраф:

$$Loss = Loss_{PPO} + \beta KL(\pi_{\theta} \parallel \pi_{ref})$$

Без reference model модель может начать «ломаться»: выдавать странные ответы, переоптимизироваться под reward и терять качество языка.

10. Преимущества vLLM перед базовым Hugging Face inference. vLLM эффективнее работает с KV-cache благодаря PagedAttention. Также он поддерживает быстрый батчинг запросов, лучше использует GPU и обычно даёт выше throughput при большом числе пользователей.

0.18 3. Практико-ориентированное задание

```
import torch.nn as nn

# Мини-словарь
src_vocab = {"<pad>": 0, "<sos>": 1, "<eos>": 2, "i": 3, "love": 4, "nlp": 5}
tgt_vocab = {"<pad>": 0, "<sos>": 1, "<eos>": 2, "я": 3, "люблю": 4, "нлп": 5}
id2tgt = {v: k for k, v in tgt_vocab.items()}

SRC = len(src_vocab)
TGT = len(tgt_vocab)
EMB = 16
H = 32

class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.emb = nn.Embedding(SRC, EMB)
        self.gru = nn.GRU(EMB, H, batch_first=True, bidirectional=True)
        self.proj = nn.Linear(2 * H, H)

    def forward(self, x):
        x = self.emb(x)
        out, h = self.gru(x)          # h: (2, B, H)
        h_fwd = h[-2]                 # последнее состояние прямого GRU
        h_bwd = h[-1]                 # последнее состояние обратного GRU
        h0_dec = torch.tanh(self.proj(torch.cat([h_fwd, h_bwd], dim=1)))
        return out, h0_dec.unsqueeze(0)

class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.emb = nn.Embedding(TGT, EMB)
        self.gru = nn.GRU(EMB, H, batch_first=True)
        self.fc = nn.Linear(H, TGT)

    def forward(self, token, h):
        x = self.emb(token)
        out, h = self.gru(x, h)
        logits = self.fc(out[:, -1])
        return logits, h

encoder = Encoder()
decoder = Decoder()
```



```

# 1. Входная фраза: "i love nlp"
src = torch.tensor([[src_vocab["i"], src_vocab["love"], src_vocab["nlp"],
    src_vocab["<eos>"]]])

# 2. Кодирование входной фразы энкодером
enc_out, dec_h = encoder(src)

# 3. Пошаговая генерация декодером
cur = torch.tensor([[tgt_vocab["<sos>"]]])
result = []

max_len = 10
for _ in range(max_len):
    logits, dec_h = decoder(cur, dec_h)
    next_id = logits.argmax(dim=1).item()

    # 4. Условие остановки
    if next_id == tgt_vocab["<eos>"]:
        break

    result.append(id2tgt[next_id])
    cur = torch.tensor([[next_id]])

print("Перевод:", " ".join(result))

```

0.18.1 Формула итогового представления токена в BiRNN

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]$$

где $\overrightarrow{h_t}$ — скрытое состояние прямой RNN, которая читает последовательность слева направо, а $\overleftarrow{h_t}$ — состояние обратной RNN, которая читает справа налево. В отличие от однонаправленной сети, BiRNN учитывает и левый, и правый контекст токена.

0.18.2 Почему GRU быстрее LSTM

GRU обычно быстрее LSTM, потому что у GRU меньше вентиляй и меньше параметров. В LSTM есть отдельное состояние памяти (c_t) и несколько ворот, а в GRU используются в основном update gate и reset gate, поэтому вычислений меньше при той же размерности hidden state.

0.19 Вариант 17

0.19.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.20 2. Вопросы с открытым ответом

1. Два следствия закона Ципфа, важных для NLP-моделей. В языке есть небольшой набор очень частых слов и огромный «длинный хвост» редких слов. Поэтому частые стоп-слова часто удаляют или понижают их вес, а редкие слова требуют специальных методов: сглаживания, subword-токенизации, fastText, BPE.

2. Допущение Маркова n-го порядка в языковой модели. Допущение Маркова говорит, что вероятность следующего слова зависит не от всей истории, а только от последних (n) слов:

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-n}, \dots, w_{i-1})$$

Это упрощает языковую модель, потому что не нужно хранить весь предыдущий контекст.

3. MLE-оценка вероятности биграммы.

$$P_{MLE}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

где $C(w_{i-1}, w_i)$ — сколько раз биграмма встретилась в корпусе, а $C(w_{i-1})$ — сколько раз слово (w_{i-1}) было первым словом биграммы.

4. Три основных гейта LSTM и их роль. В LSTM есть forget gate, input gate и output gate. Forget gate решает, что забыть из старой памяти, input gate — что записать в память, output gate — какую часть памяти выдать наружу. Общая формула гейта:

$$g_t = \sigma(W_g x_t + U_g h_{t-1} + b_g)$$

5. nn.RNN, batch_first=True, hidden_size=N, вход (B, T, D). Формы output и h_n. Для однослойной однонаправленной RNN:

$$output : (B, T, H), \quad h_n : (1, B, H)$$

output содержит скрытые состояния для всех токенов, а h_n — последнее скрытое состояние.

6. Сущности в запросе «Забронируйте столик в ресторане Утка на завтра на двоих». Интент: бронирование_столика. Сущности: ресторан = Утка, дата = завтра, количество_людей = двое.

7. Для чего применяется квантизация весов? Квантизация уменьшает размер модели, расход памяти и ускоряет инференс. Примеры методов: fp32 → fp16/bfloat16, int8/int4, симметричная и асимметричная квантизация.

8. Когда симметричная квантизация уступает асимметричной? Симметричная квантизация хуже, когда распределение весов или активаций сильно смещено относительно нуля. Например, если значения лежат в диапазоне $[2; 10]$, симметричная шкала тратит часть уровней на отрицательные числа, которых вообще нет.

9. Минимум два преимущества адаптеров перед полным fine-tuning. Адаптеры обучают только небольшие дополнительные слои, поэтому требуют меньше памяти и вычислений. Ещё они не меняют основную модель, поэтому снижается риск катастрофического забывания и проще переключаться между задачами.

10. На каком свойстве матриц весов основана LoRA? LoRA основана на предположении, что изменение весов при дообучении имеет низкий ранг. Поэтому вместо полного обновления ΔW обучаются две маленькие матрицы:

$$W' = W + \Delta W, \quad \Delta W = AB$$
$$A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times k}, \quad r \ll d, k$$

0.21 3. Практико-ориентированное задание

```
import random
import torch
import torch.nn as nn
import torch.optim as optim
from collections import Counter
```

```

# Небольшой пример корпуса
texts = [
    "кошка сидит на окне",
    "собака бегает во дворе",
    "кошка и собака любят дом",
    "машинное обучение изучает тексты",
    "редкие слова встречаются редко"
]

def tokenize(text):
    text = text.lower()
    text = re.sub(r"[^а-яёа-з ]", " ", text)
    return text.split()

tokens = []
for text in texts:
    tokens.extend(tokenize(text))

# 1. Словарь
counts = Counter(tokens)
vocab = list(counts.keys())
word2id = {w: i for i, w in enumerate(vocab)}
id2word = {i: w for w, i in word2id.items()}
ids = [word2id[w] for w in tokens]

# 2. Пары "центральное слово – слово из окна"
window = 2
pairs = []

for i, center in enumerate(ids):
    left = max(0, i - window)
    right = min(len(ids), i + window + 1)

    for j in range(left, right):
        if i != j:
            pairs.append((center, ids[j]))

# 3. Распределение для negative sampling: unigram0.75
freqs = torch.tensor([counts[id2word[i]] for i in range(len(vocab))], dtype=torch.float)
neg_probs = freqs ** 0.75
neg_probs = neg_probs / neg_probs.sum()

def sample_negatives(k):
    return torch.multinomial(neg_probs, k, replacement=True)

# 4. Skip-gram with Negative Sampling
class SGNS(nn.Module):
    def __init__(self, vocab_size, emb_dim):
        super().__init__()
        self.in_emb = nn.Embedding(vocab_size, emb_dim)
        self.out_emb = nn.Embedding(vocab_size, emb_dim)

    def forward(self, center, pos_context, neg_context):
        v_c = self.in_emb(center)           # (B, D)
        v_pos = self.out_emb(pos_context)    # (B, D)
        v_neg = self.out_emb(neg_context)    # (B, K, D)

```

```

pos_score = torch.sum(v_c * v_pos, dim=1)
pos_loss = torch.log(torch.sigmoid(pos_score))

neg_score = torch.bmm(v_neg, v_c.unsqueeze(2)).squeeze(2)
neg_loss = torch.sum(torch.log(torch.sigmoid(-neg_score)), dim=1)

loss = -(pos_loss + neg_loss).mean()
return loss

model = SGNS(vocab_size=len(vocab), emb_dim=32)
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 5. Обучение
batch_size = 4
neg_k = 5

for epoch in range(10):
    random.shuffle(pairs)
    total_loss = 0

    for i in range(0, len(pairs), batch_size):
        batch = pairs[i:i + batch_size]

        center = torch.tensor([p[0] for p in batch])
        pos_context = torch.tensor([p[1] for p in batch])
        neg_context = torch.stack([sample_negatives(neg_k) for _ in batch])

        loss = model(center, pos_context, neg_context)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"epoch={epoch+1}, loss={total_loss:.4f}")

```

0.21.1 Формула negative sampling

$$L = - \left[\log \sigma(v_c^T v_o) + \sum_{i=1}^k \log \sigma(-v_c^T v_{n_i}) \right]$$

где (v_c) — вектор центрального слова, (v_o) — вектор настоящего контекстного слова, $(v_{\{n_i\}})$ — вектор отрицательного слова, (k) — число отрицательных примеров.

0.21.2 Вероятность выбора отрицательного примера

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{u \in V} f(u)^{3/4}}$$

где $(f(w))$ — частота слова в корпусе. Степень $(3/4)$ сглаживает распределение: частые слова всё ещё выбираются чаще, но не доминируют слишком сильно.

0.21.3 Что будет с редкими словами при случайных отрицательных примерах

Если брать отрицательные слова полностью случайно, качество представлений редких слов обычно ухудшится. В корпусе с длинным хвостом будет много слишком лёгких или шумных отри-

цательных примеров, а редкие слова и так получают мало положительных обновлений. Поэтому используют распределение $(f(w)^{\frac{3}{4}})$, которое делает negative sampling более сбалансированным.

0.22 Вариант 18

0.22.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.23 2. Вопросы с открытым ответом

1. Тип семантического отношения для пар. собака — животное — гипонимия/гиперонимия: собака является видом животного. горячий — холодный — антонимия. кофеин — капучино — отношение часть-целое/компонент-объект, потому что кофеин может быть компонентом напитка. врач — доктор — синонимия.

2. Дистрибутивная гипотеза и методы на её основе. Дистрибутивная гипотеза: слова, которые встречаются в похожих контекстах, имеют похожие значения. На ней основаны матрицы совместной встречаемости, SVD/LSA, word2vec, GloVe, fastText.

3. Словарь ($|V|=2000$), окно ± 2 . Форма матрицы совместной встречаемости. Матрица будет иметь форму:

$$2000 \times 2000$$

Ячейка (C_{ij}) показывает, сколько раз слово (w_j) встретилось в окне ± 2 рядом со словом (w_i). Например, если в тексте «кошка поймала мышь» при окне 1 слово «мышь» стоит рядом с «поймала», то соответствующая ячейка увеличится на 1.

4. Зачем в word2vec применяется negative sampling и что он заменяет? Negative sampling заменяет полный softmax по всему словарю, который слишком дорогой при большом словаре. Вместо этого модель отличает настоящую пару «слово-контекст» от нескольких случайных отрицательных примеров:

$$\log \sigma(v_c^T v_w) + \sum_{i=1}^k \log \sigma(-v_{n_i}^T v_w)$$

где (v_w) — вектор центрального слова, (v_c) — правильный контекст, (v_{n_i}) — отрицательные слова.

5. Эффект «бутылочного горлышка» в seq2seq на базе RNN. В классическом seq2seq всё входное предложение сжимается в один вектор последнего скрытого состояния encoder. Из-за этого на длинных последовательностях модель может забывать начало текста и терять важную информацию.

6. Роль матрицы Value (V) в Attention. Формула внимания:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

(Q) и (K) определяют веса внимания, то есть на что смотреть, а (V) содержит сами значения, которые смешиваются по этим весам.

7. На основе чего обучается Reward Model в RLHF? Reward Model обучается на человеческих предпочтениях: человеку показывают несколько ответов на один запрос, и он выбирает лучший. Пример данных: prompt: «Объясни RNN», chosen: «RNN — сеть для последовательно-стей», rejected: «RNN — обычная нейросеть без контекста».

8. Зачем в RLHF через PPO используется копия исходной base model? Копия base model используется как reference model, чтобы новая модель не слишком далеко уходила от исходного

поведения. Обычно добавляют KL-штраф:

$$L = L_{PPO} + \beta KL(\pi_{\theta} \parallel \pi_{ref})$$

Без этого модель может переоптимизироваться под reward и начать выдавать странные или неестественные ответы.

9. Основная задача самообучения на этапе pre-training генеративных LLM. Главная задача — предсказание следующего токена по предыдущему контексту:

$$P(w_t | w_1, \dots, w_{t-1})$$

Например, по фразе «Сегодня хорошая...» модель должна предсказать вероятное продолжение: «погода».

10. Этапы документов в базовом RAG-пайплайне перед добавлением в prompt. Документы очищаются, разбиваются на chunks, затем для фрагментов строятся embeddings и сохраняются в векторную базу. После запроса находятся top-k близких фрагментов, при необходимости ранжируются, и добавляются в prompt как контекст.

0.24 3. Практико-ориентированное задание

```
import torch.nn as nn
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence

# 1. Отзывы разной длины
reviews = [
    [1, 5, 8, 2],          # "товар очень хороший"
    [1, 7, 2],             # "товар плохой"
    [1, 6, 9, 10, 2]       # "товар понравился очень сильно"
]

labels = torch.tensor([1, 0, 1]) # 1 - positive, 0 - negative

# Длины без padding
lengths = torch.tensor([len(x) for x in reviews])

# Padding до одной длины
reviews = [torch.tensor(x) for x in reviews]
batch = pad_sequence(reviews, batch_first=True, padding_value=0)

# 2. Модель: Embedding + LSTM + classifier
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_dim, num_classes):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            input_size=emb_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x, lengths):
        x = self.emb(x) # (B, T, E)

        packed = pack_padded_sequence(
            x,
```

```

        lengths.cpu(),
        batch_first=True,
        enforce_sorted=False
    )

    output, (h_n, c_n) = self.lstm(packed)

    # 3. Итоговое представление отзыва – последнее hidden state
    final_repr = h_n[-1] # (B, H)
    logits = self.fc(final_repr) # (B, num_classes)

    return logits

model = SentimentLSTM(
    vocab_size=20,
    emb_dim=16,
    hidden_dim=32,
    num_classes=2
)

logits = model(batch, lengths)
pred = logits.argmax(dim=1)

print("logits:", logits)
print("predicted classes:", pred)

```

0.24.1 Формула обновления долговременной памяти LSTM

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

где (c_t) — новая долговременная память, (c_{t-1}) — старая память, (f_t) — forget gate, который решает, что забыть, (i_t) — input gate, который решает, что записать, $\tilde{c}_t$ — кандидат новой информации, $\odot$ — поэлементное умножение.

Главный элемент LSTM, который помогает градиентам проходить через много шагов, — это ячейка памяти (c_t) с аддитивным обновлением. Если $f_t \approx 1$, то старая память почти напрямую переносится дальше, и градиент не затухает так быстро, как в базовой RNN.

0.25 Вариант 7

0.25.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.26 2. Вопросы с открытым ответом

1. TF-IDF: слово встречается 3 раза в документе из 30 термов, коллекция 1000 документов, слово есть в 10 документах.

$$TF = \frac{3}{30} = 0.1$$

$$IDF = \log_{10} \frac{1000}{10} = \log_{10} 100 = 2$$

$$TF-IDF = TF \cdot IDF = 0.1 \cdot 2 = 0.2$$

2. Минимум два преимущества fastText перед word2vec. fastText учитывает не только слово целиком, но и символьные n-граммы, то есть части слова. Поэтому он лучше работает с редкими словами, словами с ошибками и словами вне словаря.

3. Формула residual connection и какую проблему решает. Стандартная формула:

$$y = F(x) + x$$

где (x) — вход слоя, (F(x)) — преобразование слоя. Residual connection помогает обучать глубокие сети и уменьшает проблему затухающего градиента.

4. Если (W_Q) и (W_K) нулевые, а (V) вычисляется обычно, чему равен (Z=Attention(Q,K,V))? Формула:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Если (Q=0) и (K=0), то (QK^T=0), а softmax по нулевым значениям даёт равномерные веса. Значит, выход (Z) будет усреднением строк (V), то есть каждый токен получит примерно одинаковую смесь значений.

5. Как формулируется задача Next Sentence Prediction в BERT? NSP — это бинарная задача: определить, является ли второе предложение реальным продолжением первого. Пример: A = "Я пришёл домой.", B = "Я включил компьютер." → IsNext; если B = "Кошка спит на диване." из другого текста → NotNext.

6. Сущности в запросе «Забронируйте столик в ресторане Утка на завтра на двоих». Интент: бронирование_столика. Сущности: ресторан = Утка, тип — название ресторана; дата = завтра, тип — дата; количество_людей = двое, тип — число гостей.

7. Что означает SFT в схеме обучения LLM? На каких данных проводится? SFT — это Supervised Fine-Tuning, то есть дообучение с учителем. Оно проводится на размеченных данных вида «инструкция/вопрос → правильный ответ», например на диалогах, инструкциях и качественных эталонных ответах.

8. Минимум два преимущества адаптеров перед полным fine-tuning. Адаптеры обучают только небольшие дополнительные слои, поэтому требуют меньше памяти и вычислений. Также они не меняют основную модель, поэтому меньше риск катастрофического забывания и проще переключаться между задачами.

9. Слой (W) размерности 1024×1024. Сколько параметров добавит LoRA при (r=8)? LoRA добавляет две матрицы:

$$A \in \mathbb{R}^{1024 \times 8}, \quad B \in \mathbb{R}^{8 \times 1024}$$

$$1024 \cdot 8 + 8 \cdot 1024 = 8192 + 8192 = 16384$$

Итого: **16 384 обучаемых параметра.**

10. Этапы документов в базовом RAG-пайплайне перед добавлением в prompt. Документы очищают, разбивают на фрагменты/chunks, затем для них строят embeddings и сохраняют в векторную базу. После запроса находят top-k похожих фрагментов, при необходимости ранжируют, и добавляют их в prompt как внешний контекст.

0.27 3. Практико-ориентированное задание

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

docs = [
```



```

    "Кошка поймала мышь",
    "Собака увидела кошку",
    "Кошка и собака живут дома"
]

stop_words = {"и", "в", "на", "а", "но"}

def tokenize(text):
    text = text.lower()
    text = re.sub(r"[^a-яёa-z ]", " ", text)
    return [w for w in text.split() if w not in stop_words]

# 1. Предобработка корпуса
corpus = [tokenize(doc) for doc in docs]

# 2. Формирование словаря
vocab = sorted(set(word for doc in corpus for word in doc))
word2id = {word: i for i, word in enumerate(vocab)}

# 3. Матрица совместной встречаемости с окном ±w
w = 2
C = np.zeros((len(vocab), len(vocab)), dtype=int)

for doc in corpus:
    for i, center_word in enumerate(doc):
        center_id = word2id[center_word]

        left = max(0, i - w)
        right = min(len(doc), i + w + 1)

        for j in range(left, right):
            if i == j:
                continue

            context_word = doc[j]
            context_id = word2id[context_word]
            C[center_id, context_id] += 1

cooc_df = pd.DataFrame(C, index=vocab, columns=vocab)
print("Матрица совместной встречаемости:")
print(cooc_df)

# Дополнительно: bag of words для документов
vectorizer = CountVectorizer(tokenizer=tokenize, token_pattern=None)
bow = vectorizer.fit_transform(docs)

bow_df = pd.DataFrame(
    bow.toarray(),
    columns=vectorizer.get_feature_names_out()
)

print("\nBag of Words:")
print(bow_df)

```

0.27.1 Формула матрицы совместной встречаемости

$$C_{ij} = \text{count}(w_i, w_j)$$

где (C_{ij}) — сколько раз слово (w_j) встретилось рядом со словом (w_i) в окне размера (w) . Если окно $\pm w$, то для каждого центрального слова берутся до (w) слов слева и до (w) слов справа.

Стоп-слова и слишком частые токены лучше удалять до заполнения матрицы, потому что они встречаются почти со всеми словами и засоряют статистику. Из-за них модель хуже выделяет реальные смысловые связи между словами.

0.28 Вариант 29

0.28.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.29 2. Вопросы с открытым ответом

1. Формула закона Ципфа и что такое ранг.

$$f(w) \approx \frac{C}{r(w)^\alpha}, \quad \alpha \approx 1$$

где $(f(w))$ — частота слова, $(r(w))$ — ранг слова по частоте, (C) — константа. Ранг 1 имеет самое частое слово, ранг 2 — второе по частоте и т.д.

2. Два применения закона Ципфа в NLP. Закон Ципфа помогает понимать, что в языке есть немного очень частых слов и много редких. Поэтому его используют для удаления/понижения веса стоп-слов и для обработки редких слов: сглаживание, subword-токенизация, fastText, BPE.

3. Допущение Маркова n-го порядка. Допущение Маркова: следующее слово зависит не от всей истории, а только от последних (n) слов:

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-n}, \dots, w_{i-1})$$

Это упрощает языковую модель и лежит в основе n-грамм.

4. Веса RNN на разных временных шагах. В RNN на всех временных шагах используются одни и те же веса.

$$\begin{aligned} h_t &= \tanh(W_x x_t + W_h h_{t-1} + b) \\ h_{t+1} &= \tanh(W_x x_{t+1} + W_h h_t + b) \end{aligned}$$

Это позволяет одной ячейкой обрабатывать последовательности разной длины.

5. Роль матрицы Value (V) в Attention.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

(Q) и (K) задают веса внимания, то есть на что смотреть, а (V) содержит сами значения, которые смешиваются по этим весам.

6. Cross-attention и self-attention. В self-attention (Q, K, V) берутся из одной последовательности: например, слова предложения смотрят друг на друга. В cross-attention (Q) берётся из decoder, а (K, V) — из encoder, например в машинном переводе decoder обращается к исходному предложению.

7. Slot filling в диалоговой системе. Slot filling — это извлечение конкретных параметров из реплики пользователя для выполнения интента. Например: «Забронируй столик в ресторане Утка завтра на двоих» → restaurant=Утка, date=завтра, people=2.

8. Инициализация матриц A и B в LoRA. Обычно (A) инициализируется маленькими случайными значениями, а (B) — нулями. Тогда в начале:

$$\Delta W = AB = 0$$

и модель сначала ведёт себя как исходная предобученная модель.

9. Почему квантизация часто применяется асимметрично? Асимметричная квантизация лучше подходит, когда значения не центрированы около нуля, например активации часто только положительные. Методы: int8/int4, fp16/bfloat16, симметричная и асимметричная квантизация, per-tensor/per-channel, PTQ и QAT.

10. Когда симметричная квантизация уступает асимметричной? Когда диапазон значений сильно смещён относительно нуля. Например, если значения лежат в $([2, 10])$, симметричная шкала будет тратить уровни на отрицательную область, которой нет, а асимметричная лучше использует весь диапазон.

0.30 3. Практико-ориентированное задание

```
import torch.nn as nn
import torch.optim as optim
from collections import Counter

corpus = [
    ["я", "люблю", "машинное", "обучение"],
    ["я", "изучаю", "обработку", "текста"],
    ["машинное", "обучение", "полезно"]
]

# 1. Словарь и кодирование токенов
tokens = [w for sent in corpus for w in sent]
vocab = sorted(set(tokens))
word2id = {w: i for i, w in enumerate(vocab)}
id2word = {i: w for w, i in word2id.items()}
encoded = [[word2id[w] for w in sent] for sent in corpus]

# 2. Обучающие пары: контекст -> центральное слово
window = 2
pairs = []

for sent in encoded:
    for i, center in enumerate(sent):
        ctx = []
        for j in range(max(0, i - window), min(len(sent), i + window + 1)):
            if i != j:
                ctx.append(sent[j])
        if ctx:
            pairs.append((ctx, center))

# 3. Два embedding-слоя: входной и выходной
class CBOWNegativeSampling(nn.Module):
    def __init__(self, vocab_size, emb_dim):
        super().__init__()
        self.in_emb = nn.Embedding(vocab_size, emb_dim)
        self.out_emb = nn.Embedding(vocab_size, emb_dim)

    def forward(self, context_ids, target_id, negative_ids):
        # усреднённый вектор контекста
        ctx_vec = self.in_emb(context_ids).mean(dim=0)  # (D,)

        pos_vec = self.out_emb(target_id)  # (D,)
        neg_vec = self.out_emb(negative_ids)  # (K, D)
```

```

pos_score = torch.dot(ctx_vec, pos_vec)
neg_score = torch.mv(neg_vec, ctx_vec)

loss = -(
    torch.log(torch.sigmoid(pos_score)) +
    torch.log(torch.sigmoid(-neg_score)).sum()
)

return loss

model = CBOWNegativeSampling(vocab_size=len(vocab), emb_dim=32)
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Распределение для negative sampling
counts = Counter(tokens)
freqs = torch.tensor([counts[id2word[i]] for i in range(len(vocab))], dtype=torch.float)
neg_probs = freqs ** 0.75
neg_probs = neg_probs / neg_probs.sum()

# 4-5. Прямой проход и обновление параметров через backprop
neg_k = 5

for epoch in range(10):
    total_loss = 0

    for ctx, center in pairs:
        context_ids = torch.tensor(ctx)
        target_id = torch.tensor(center)
        negative_ids = torch.multinomial(neg_probs, neg_k, replacement=True)

        loss = model(context_ids, target_id, negative_ids)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    total_loss += loss.item()

    print(f"epoch={epoch+1}, loss={total_loss:.4f}")

```

0.30.1 Формула обучения центрального слова по контексту

$$L = - \left[\log \sigma(u_w^T h) + \sum_{i=1}^k \log \sigma(-u_{n_i}^T h) \right]$$

где (h) — усреднённый вектор слов контекста из входной embedding-матрицы, (u_w) — выходной вектор правильного центрального слова, (u_{n_i}) — выходные векторы отрицательных слов.

Если увеличить размер контекстного окна, модель начинает лучше ловить тематическую/семантическую близость слов. Малое окно чаще даёт синтаксические связи, потому что учитывает ближайших соседей слова.

0.31 Вариант 20

0.31.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.32 2. Вопросы с открытым ответом

1. Чем инициализируется скрытое состояние RNN на первом шаге? Обычно начальное скрытое состояние задают нулевым вектором:

$$h_0 = 0$$

Потому что на первом шаге у модели ещё нет предыдущего контекста. Формула RNN cell:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

2. Формула обновления (c_t) в LSTM. Что будет, если ($f_t=0$)?

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

где (f_t) — forget gate, (i_t) — input gate, $\tilde{c}_t$ — кандидат новой памяти. Если ($f_t=0$), старая память (c_{t-1}) полностью забывается.

3. Формула residual connection и какую проблему решает.

$$y = F(x) + x$$

где (x) — вход, ($F(x)$) — преобразование слоя. Residual connection помогает обучать глубокие сети и уменьшает проблему затухающего градиента.

4. Если (W_Q) и (W_K) нулевые, чему равен ($\text{Attention}(Q, K, V)$)?

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Если ($Q=0$) и ($K=0$), то ($QK^T=0$). softmax по нулям даёт равномерные веса, значит результат будет усреднением значений (V).

5. Как формулируется Next Sentence Prediction в BERT? NSP — бинарная задача: определить, является ли второе предложение настоящим продолжением первого. Пример: «Я пришёл домой.» + «Я включил компьютер.» → IsNext; «Я пришёл домой.» + «Слон живёт в Африке.» → NotNext.

6. Интент в логике чат-бота. Интент — это намерение пользователя, то есть цель его сообщения. Примеры: «Какая завтра погода?» → weather_request; «Закажи пиццу» → food_order; «Отмени бронь» → cancel_booking.

7. Для чего применяется квантизация весов? Квантизация уменьшает размер модели, расход памяти и ускоряет инференс. Методы: fp32 → fp16/bfloat16, int8/int4, симметричная и асимметричная квантизация.

8. Когда симметричная квантизация уступает асимметричной? Когда значения сильно смещены относительно нуля. Например, если активации лежат в диапазоне $([2; 10])$, симметричная шкала тратит уровни на отрицательные значения, которых нет.

9. Минимум два преимущества адаптеров перед полным fine-tuning. Адаптеры обучают только небольшие дополнительные слои, поэтому требуют меньше памяти и вычислений. Также они не меняют основную модель, уменьшают риск катастрофического забывания и позволяют быстро переключаться между задачами.

10. Что даёт bfloat16 по сравнению с float16? В bfloat16 больше бит отведено под экспоненту, поэтому он поддерживает более широкий диапазон чисел. Это делает обучение стабильнее: меньше риск overflow/underflow, хотя точность мантииссы ниже, чем у float16.

0.33 3. Практико-ориентированное задание

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import normalize
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Корпус новостных документов
docs = [
    "Правительство обсудило новые меры поддержки экономики",
    "Центробанк сообщил об изменении ключевой ставки",
    "Футбольная команда выиграла важный матч чемпионата",
    "Ученые разработали новую модель искусственного интеллекта",
    "Спортсмен установил новый мировой рекорд"
]

# 1. Матрица документ-термин с TF-IDF весами
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)  # shape: документы x термины

# 2. Усечённое SVD-разложение
k_dim = 2
svd = TruncatedSVD(n_components=k_dim, random_state=42)
X_k = svd.fit_transform(X)  # документы в k-мерном пространстве
X_k = normalize(X_k)

# 3. Проекция нового запроса и выдача top-k
query = "новости экономики и ключевая ставка"
q = vectorizer.transform([query])
q_k = normalize(svd.transform(q))

top_k = 3
scores = cosine_similarity(q_k, X_k).ravel()
best_ids = np.argsort(scores)[::-1][:top_k]

for i in best_ids:
    print(f"score={scores[i]:.3f} | {docs[i]}")
```

0.33.1 Формула усечённого SVD-разложения

$$X \approx U_k \Sigma_k V_k^T$$

где (X) — TF-IDF матрица, (U_k) и (V_k^T) — матрицы скрытых факторов, Σ_k — диагональная матрица (k) главных сингулярных значений.

Усечённое SVD лучше полного для разреженной TF-IDF матрицы, потому что оставляет только главные скрытые темы, уменьшает размерность, экономит память и отбрасывает шум.

0.34 Вариант 40

0.34.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.35 2. Вопросы с открытым ответом

1. TF-IDF: слово встречается 3 раза в документе из 30 термов, коллекция 1000 документов, слово есть в 10 документах.

$$TF = \frac{3}{30} = 0.1$$

$$IDF = \log_{10} \frac{1000}{10} = \log_{10} 100 = 2$$

$$TF-IDF = TF \cdot IDF = 0.1 \cdot 2 = 0.2$$

2. Минимум два преимущества fastText перед word2vec. fastText учитывает не только слово целиком, но и символьные n-граммы, то есть части слова. Поэтому он лучше работает с редкими словами, словами с опечатками и словами вне словаря.

3. Формула residual connection и какую проблему решает.

$$y = F(x) + x$$

где (x) — вход слоя, (F(x)) — преобразование слоя. Residual connection помогает обучать глубокие сети и уменьшает проблему затухающего градиента.

4. Если (W_Q) и (W_K) нулевые, а (V) вычисляется обычно, чему равен (Z=Attention(Q,K,V))?

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Если (Q=0) и (K=0), то (QK^T=0). softmax по нулям даёт равномерные веса, значит (Z) будет усреднением значений (V).

5. Как формулируется Next Sentence Prediction в BERT? NSP — это бинарная задача: определить, является ли второе предложение настоящим продолжением первого. Например: «Я пришёл домой.» + «Я включил компьютер.» → IsNext; «Я пришёл домой.» + «Слон живёт в Африке.» → NotNext.

6. Сущности в запросе «Забронируйте столик в ресторане Утка на завтра на двоих». Интент: бронирование_столика. Сущности: ресторан = Утка, тип — название ресторана; дата = завтра, тип — дата; количество_людей = двое, тип — число гостей.

7. Что означает SFT в схеме обучения LLM? SFT — это Supervised Fine-Tuning, то есть дообучение с учителем. Оно проводится на размеченных данных формата «инструкция/вопрос → правильный ответ», например на диалогах и эталонных ответах.

8. Минимум два преимущества адаптеров перед полным fine-tuning. Адаптеры обучают только небольшие дополнительные слои, поэтому требуют меньше памяти и вычислений. Также они не меняют основную модель, снижают риск катастрофического забывания и позволяют быстро переключаться между задачами.

9. Слой (W) размерности 1024×1024. Сколько параметров добавит LoRA при (r=8)? LoRA добавляет две матрицы:

$$A \in \mathbb{R}^{1024 \times 8}, \quad B \in \mathbb{R}^{8 \times 1024}$$

$$1024 \cdot 8 + 8 \cdot 1024 = 8192 + 8192 = 16384$$

Итого: **16 384 обучаемых параметра.**

10. Этапы документов в базовом RAG-пайплайне перед добавлением в prompt. Документы очищают, разбивают на фрагменты/chunks, затем для них строят embeddings и сохраняют в векторную базу. После запроса находят top-k похожих фрагментов, при необходимости ранжируют, и добавляют их в prompt как внешний контекст.

0.36 3. Практико-ориентированное задание

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

docs = [
    "Кошка поймала мышь",
    "Собака увидела кошку",
    "Кошка и собака живут дома"
]

stop_words = {"и", "в", "на", "а", "но"}

def tokenize(text):
    text = text.lower()
    text = re.sub(r"^[а-яёа-з ]", " ", text)
    return [w for w in text.split() if w not in stop_words]

# 1. Предобработка корпуса
corpus = [tokenize(doc) for doc in docs]

# 2. Формирование словаря
vocab = sorted(set(word for doc in corpus for word in doc))
word2id = {word: i for i, word in enumerate(vocab)}

# 3. Матрица совместной встречаемости с окном ±w
w = 2
C = np.zeros((len(vocab), len(vocab)), dtype=int)

for doc in corpus:
    for i, center_word in enumerate(doc):
        center_id = word2id[center_word]

        left = max(0, i - w)
        right = min(len(doc), i + w + 1)

        for j in range(left, right):
            if i == j:
                continue

            context_word = doc[j]
            context_id = word2id[context_word]
            C[center_id, context_id] += 1

cooc_df = pd.DataFrame(C, index=vocab, columns=vocab)
print("Матрица совместной встречаемости:")
print(cooc_df)

# Bag of Words для документов
vectorizer = CountVectorizer(tokenizer=tokenize, token_pattern=None)
bow = vectorizer.fit_transform(docs)

bow_df = pd.DataFrame(
    bow.toarray(),
    columns=vectorizer.get_feature_names_out()
)
```



```
print("\nBag of Words:")
print(bow_df)
```

0.36.1 Формула матрицы совместной встречаемости

$$C_{ij} = \text{count}(w_i, w_j)$$

где (C_{ij}) — сколько раз слово (w_j) встретилось рядом со словом (w_i) в окне размера (w) . Если окно $\pm w$, то для каждого центрального слова берутся до (w) слов слева и до (w) слов справа.

Стоп-слова и слишком частые токены лучше удалять до заполнения матрицы, потому что они встречаются почти со всеми словами и засоряют статистику. Из-за них модель хуже выделяет реальные смысловые связи между словами.

0.37 Вариант 6

0.37.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.38 2. Вопросы с открытым ответом

1. Формула перплексии через перекрёстную энтропию (H).

$$PPL = e^H$$

Если энтропия считается по логарифму 2, то:

$$PPL = 2^H$$

Перплексия показывает, насколько модель «удивляется» тексту: чем выше вероятность правильного текста, тем ниже (H) и тем ниже перплексия.

2. Базовая единица WordNet и типы отношений. Базовая единица WordNet — синсет, то есть набор слов-синонимов, обозначающих один смысл. Между синсетами бывают отношения: гипонимия/гиперонимия, меронимия/холонимия, синонимия, антонимия.

3. Тип семантического отношения для пар. собака — животное — гипонимия/гиперонимия, собака является видом животного. колесо — автомобиль — меронимия/холонимия, колесо является частью автомобиля. большой — маленький — антонимия. врач — доктор — синонимия.

4. Почему Skip-gram — это self-supervised learning? В Skip-gram не нужны ручные метки: обучающие пары автоматически берутся из самого текста. Центральное слово является входом, а слова из его контекстного окна становятся «метками» для обучения.

5. Функция активации в базовой RNN. В базовой RNN обычно используется tanh:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

где (x_t) — вход на шаге (t) , (h_{t-1}) — прошлое скрытое состояние, (h_t) — новое скрытое состояние.

6. Последствия затухания/взрыва градиентов в RNN. При затухании градиента RNN плохо запоминает дальние зависимости и почти не обучает ранние шаги последовательности. При взрыве градиента обучение становится нестабильным: loss может резко расти, веса становятся слишком большими.

7. Зачем в RLHF через PPO нужна копия base model? Копия исходной модели используется как reference model, чтобы новая политика не слишком сильно отклонялась от исходной модели. Обычно добавляют KL-штраф:

$$L = L_{PPO} + \beta KL(\pi_\theta || \pi_{ref})$$

Без неё модель может переоптимизироваться под reward и начать выдавать странные, неестественные ответы.

8. Основная задача pre-training генеративных LLM. Основная задача — предсказание следующего токена по предыдущему контексту:

$$P(w_t | w_1, \dots, w_{t-1})$$

Например, по фразе «Сегодня хорошая...» модель должна предсказать вероятное продолжение: «погода».

9. LoRA: что замораживается и что обучается? В LoRA исходные веса модели (W) замораживаются, а обучаются только маленькие матрицы (A) и (B):

$$W' = W + \Delta W, \quad \Delta W = AB$$

где (A) и (B) имеют низкий ранг, поэтому обучаемых параметров намного меньше.

10. Что даёт bfloat16 по сравнению с float16? В bfloat16 больше бит отведено под экспоненту, поэтому он поддерживает более широкий диапазон чисел. Это уменьшает риск overflow/underflow и делает обучение нейросетей стабильнее, хотя точность мантиссы ниже.

0.39 3. Практико-ориентированное задание

```
import torch
import torch.nn as nn

# Фиксированные интенты и BIO-теги сущностей
intents = ["book_doctor", "cancel_appointment", "other"]
tags = ["O", "B-SPECIALTY", "I-SPECIALTY", "B-DATE", "I-DATE", "B-TIME", "I-TIME"]

word2id = {
    "<pad>": 0, "<unk>": 1,
    "запиши": 2, "меня": 3, "к": 4, "терапевту": 5,
    "завтра": 6, "на": 7, "10": 8, "утра": 9,
    "отмени": 10, "запись": 11
}

def tokenize(text):
    text = text.lower()
    text = re.sub(r"[^a-яё0-9 ]", " ", text)
    return text.split()

def encode(tokens):
    return torch.tensor([[word2id.get(t, word2id["<unk>"]) for t in tokens]])

class NLU(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_dim, n_intents, n_tags):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, emb_dim, padding_idx=0)
        self.rnn = nn.GRU(emb_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.intent_head = nn.Linear(2 * hidden_dim, n_intents)
        self.tag_head = nn.Linear(2 * hidden_dim, n_tags)

    def forward(self, x):
        x = self.emb(x)
        out, h = self.rnn(x)
```

```

        # Представление всей фразы для intent classification
        sent_repr = out.mean(dim=1)
        intent_logits = self.intent_head(sent_repr)

        # Представления каждого токена для slot filling
        tag_logits = self.tag_head(out)

        return intent_logits, tag_logits

model = NLU(
    vocab_size=len(word2id),
    emb_dim=16,
    hidden_dim=32,
    n_intents=len(intents),
    n_tags=len(tags)
)

# Пример входной фразы
text = "Запиши меня к терапевту завтра на 10 утра"
tokens = tokenize(text)
x = encode(tokens)

intent_logits, tag_logits = model(x)

# 1. Предсказание интента
intent_id = intent_logits.argmax(dim=1).item()
intent = intents[intent_id]

# 2. Предсказание тегов сущностей
tag_ids = tag_logits.argmax(dim=2)[0].tolist()
pred_tags = [tags[i] for i in tag_ids]

# 3. Сбор структурированного результата
slots = {}
current_slot = None
current_words = []

for token, tag in zip(tokens, pred_tags):
    if tag == "0":
        if current_slot:
            slots[current_slot] = " ".join(current_words)
            current_slot, current_words = None, []
        else:
            prefix, slot = tag.split("-")
            slot = slot.lower()

            if prefix == "B":
                if current_slot:
                    slots[current_slot] = " ".join(current_words)
                    current_slot = slot
                    current_words = [token]
                else:
                    current_words.append(token)
            else:
                current_words.append(token)

if current_slot:
    slots[current_slot] = " ".join(current_words)

result = {

```

```

    "intent": intent,
    "slots": slots
}

print(result)

```

0.39.1 Формула распределения вероятностей по интендам

$$p(y = i | x) = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

где (z_i) — логит для интенда (i), (K) — число интендов. `softmax` превращает выходы классификатора в вероятности по всем интендам.

0.39.2 Почему сущности извлекают как метки токенов

Сущности удобнее извлекать как последовательность тегов, потому что слот может занимать один или несколько токенов: «10 утра», «следующий понедельник», «детский кардиолог». Поэтому используют BIO-разметку: B-DATE, I-DATE, B-TIME, а не просто выбирают одну подстроку из всей фразы.

0.40 Вариант 25

0.40.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.41 2. Вопросы с открытым ответом

1. Преимущества косинусной меры перед евклидовым расстоянием при сравнении документов. Косинусная мера сравнивает направление векторов, а не их длину, поэтому меньше зависит от размера документа. Она хорошо подходит для разреженных TF-IDF-векторов и показывает сходство по распределению слов.

2. Почему Skip-gram относят к self-supervised learning? Skip-gram не требует ручной разметки: обучающие метки берутся из самого текста. Центральное слово используется как вход, а слова из его контекстного окна становятся целевыми метками.

3. Чем инициализируется скрытое состояние RNN на первом шаге? Обычно начальное скрытое состояние задают нулевым вектором:

$$h_0 = 0$$

Формула RNN cell:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Так делают, потому что на первом шаге у модели ещё нет предыдущего контекста.

4. Как соотносятся веса RNN на разных временных шагах? В RNN на всех временных шагах используются одни и те же веса (W_x , W_h).

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

$$h_{t+1} = \tanh(W_x x_{t+1} + W_h h_t + b)$$

Это позволяет одной и той же ячейкой обрабатывать последовательности разной длины.

5. Формула обновления (c_t) в LSTM. Что будет, если ($f_t=0$)?

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

где (f_t) — forget gate, (i_t) — input gate, $\tilde{c}_t$ — кандидат новой памяти. Если ($f_t=0$), старая память (c_{t-1}) полностью забывается.

6. Slot filling в диалоговой системе. Slot filling — это извлечение параметров из реплики пользователя для выполнения интента. Например: «Запиши меня к врачу завтра в 10» → intent=запись_к_врачу, date=завтра, time=10, service=врач.

7. Особенность in-context learning по сравнению с fine-tuning. In-context learning не меняет веса модели: примеры задачи просто добавляются в prompt. Плюсы — быстро и не требует обучения; минусы — зависит от качества prompt, ограничен длиной контекста и может быть менее стабильным, чем fine-tuning.

8. На основе чего обучается Reward Model в RLHF? Reward Model обучается на человеческих предпочтениях: человеку показывают несколько ответов на один запрос, и он выбирает лучший. Пример данных:

chosen: "RNN — сеть для последовательностей, которая хранит скрытое состояние."
rejected: "RNN — это просто обычная нейросеть."

9. Зачем в RLHF через PPO используется копия исходной base model? Копия исходной модели используется как reference model, чтобы новая модель не слишком сильно отклонялась от исходного поведения. Обычно добавляют KL-штраф:

$$L = L_{PPO} + \beta KL(\pi_{\theta} \| \pi_{ref})$$

Без неё модель может переоптимизироваться под reward и начать выдавать странные ответы.

10. Что даёт Tree-of-Thought по сравнению с базовым CoT? CoT обычно строит одну цепочку рассуждений, а Tree-of-Thought рассматривает несколько ветвей решения и может выбирать лучшую. Например, при задаче на логику модель пробует несколько вариантов решения, оценивает их и отбрасывает неправильные.

0.42 3. Практико-ориентированное задание

```
import math

train = [
    "я люблю машинное обучение",
    "я люблю обработку текста",
    "машинное обучение это интересно"
]

test = [
    "я люблю обучение",
    "обработка текста это интересно"
]

def tokenize(sent):
    return ["<s>"] + sent.lower().split() + ["</s>"]

# 1. Сбор счётчиков униграмм и биграмм
unigrams = Counter()
bigrams = Counter()
vocab = set()
```

```

for sent in train:
    tokens = tokenize(sent)
    vocab.update(tokens)
    unigrams.update(tokens[:-1])
    bigrams.update(zip(tokens[:-1], tokens[1:]))

V = len(vocab)

# 2. Сглаженная вероятность биграммы, Laplace smoothing
def prob(w_prev, w):
    return (bigrams[(w_prev, w)] + 1) / (unigrams[w_prev] + V)

# 3. Подсчёт perplexity на тестовом корпусе
log_prob = 0
N = 0

for sent in test:
    tokens = tokenize(sent)
    for w_prev, w in zip(tokens[:-1], tokens[1:]):
        p = prob(w_prev, w)
        log_prob += math.log(p)
        N += 1

cross_entropy = -log_prob / N
perplexity = math.exp(cross_entropy)

print("Cross-entropy:", cross_entropy)
print("Perplexity:", perplexity)

```

0.42.1 MLE-оценка вероятности биграммы

$$P_{MLE}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

где $C(w_{i-1}, w_i)$ — число появлений биграммы, а $C(w_{i-1})$ — число появлений первого слова как контекста.

0.42.2 Связь cross-entropy и perplexity

$$H = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{i-1})$$

$$PPL = e^H$$

Перплексия показывает, насколько модель «удивляется» тестовому тексту: чем меньше perplexity, тем лучше модель предсказывает следующие слова.

Без сглаживания, если в тесте встретится биграмма, которой не было в обучении, её вероятность будет равна нулю. Тогда $\log 0$ не определён, cross-entropy и perplexity становятся бесконечными.

0.43 Вариант 3

0.43.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.44 2. Вопросы с открытым ответом

1. Последствия затухания/взрыва градиентов в базовой RNN. При затухании градиента RNN плохо запоминает дальние зависимости: ранние слова почти не влияют на результат. При взрыве градиента обучение становится нестабильным: loss резко растёт, веса становятся слишком большими.

2. Три основных гейта LSTM и их роль. В LSTM есть forget gate, input gate и output gate. Forget gate решает, что забыть из старой памяти, input gate — что записать в память, output gate — какую часть памяти выдать наружу. Общая формула гейта:

$$g_t = \sigma(W_g x_t + U_g h_{t-1} + b_g)$$

3. Почему Transformer требует positional encoding? Self-attention сам по себе не учитывает порядок слов: для него токены без позиций похожи на набор элементов. Positional encoding добавляет информацию о позиции токена, чтобы модель различала, например, «кошка поймала мышь» и «мышь поймала кошку».

4. Преимущества синусоидального positional encoding перед обучаемым. Синусоидальное кодирование не добавляет обучаемых параметров. Также оно может работать с длинами последовательностей, которых не было при обучении, потому что позиции задаются формулой.

5. Основные слои блока энкодера Transformer. Блок энкодера содержит multi-head self-attention, feed-forward network, residual connections и layer normalization. Self-attention ищет связи между токенами, FFN преобразует признаки, residual connection и normalization стабилизируют обучение.

6. Slot filling в диалоговой системе. Slot filling — это извлечение параметров из реплики пользователя для выполнения интента. Например: «Запиши меня к врачу завтра в 10» → intent=запись_к_врачу, date=завтра, time=10, service=врач.

7. На основе чего обучается Reward Model в RLHF? Reward Model обучается на человеческих предпочтениях: человеку показывают несколько ответов модели на один запрос, и он выбирает лучший. Пример данных:

chosen: "RNN — сеть для последовательностей, которая хранит скрытое состояние."
rejected: "RNN — это просто обычная нейросеть."

8. Зачем в PPO используется копия исходной base model? Копия base model используется как reference model, чтобы новая модель не слишком сильно отклонялась от исходного поведения. Обычно добавляют KL-штраф:

$$L = L_{PPO} + \beta KL(\pi_\theta \| \pi_{ref})$$

Без неё модель может переоптимизироваться под reward и начать выдавать странные ответы.

9. Основная задача pre-training генеративных LLM. Главная задача — предсказание следующего токена по предыдущему контексту:

$$P(w_t | w_1, \dots, w_{t-1})$$

Например, по фразе «Сегодня хорошая...» модель должна предсказать вероятное продолжение: «погода».

10. Что даёт Tree-of-Thought по сравнению с CoT? CoT строит одну цепочку рассуждений, а Tree-of-Thought рассматривает несколько вариантов решения как дерево. Например, в логической задаче модель пробует несколько путей, оценивает их и выбирает лучший.

0.45 3. Практико-ориентированное задание

```

import math

train = [
    "я люблю машинное обучение",
    "я люблю обработку текста",
    "машинное обучение это интересно"
]

test = [
    "я люблю обучение",
    "обработка текста это интересно"
]

def tok(s):
    return ["<s>"] + s.lower().split() + ["</s>"]

# 1. Счётчики униграмм и биграмм
unigrams = Counter()
bigrams = Counter()
vocab = set()

for sent in train:
    t = tok(sent)
    vocab.update(t)
    unigrams.update(t[:-1])
    bigrams.update(zip(t[:-1], t[1:]))

V = len(vocab)

# 2. Сглаженная вероятность биграммы
def p_bigram(w_prev, w):
    return (bigrams[(w_prev, w)] + 1) / (unigrams[w_prev] + V)

# 3. Перплексия на тесте
log_p = 0
N = 0

for sent in test:
    t = tok(sent)
    for w_prev, w in zip(t[:-1], t[1:]):
        log_p += math.log(p_bigram(w_prev, w))
        N += 1

H = -log_p / N
PPL = math.exp(H)

print("Cross-entropy:", H)
print("Perplexity:", PPL)

```

0.45.1 MLE-оценка вероятности биграммы

$$P_{MLE}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

где $C(w_{i-1}, w_i)$ — число появлений биграммы, $C(w_{i-1})$ — число появлений предыдущего слова как контекста.

0.45.2 Связь cross-entropy и perplexity

$$H = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{i-1})$$

$$PPL = e^H$$

Без сглаживания, если в тесте встретится биграмма, которой не было в обучении, её вероятность будет (0). Тогда $\log 0$ не определён, cross-entropy и perplexity становятся бесконечными.

0.46 Вариант 27

0.46.1 1. Теоретический вопрос

Не отвечаю по инструкции.

0.47 2. Вопросы с открытым ответом

1. Закон Ципфа: формула частоты слова через ранг.

$$f(w) \approx \frac{C}{r(w)^\alpha}, \quad \alpha \approx 1$$

где $f(w)$ — частота слова, $r(w)$ — ранг слова по частоте, C — константа. Самое частое слово имеет ранг 1, второе по частоте — ранг 2 и т.д.

2. Два следствия закона Ципфа для NLP. В языке есть немного очень частых слов и много редких слов. Поэтому частые слова часто удаляют или понижают их вес как стоп-слова, а редкие слова требуют специальных методов: сглаживания, subword-токенизации, fastText или BPE.

3. Минимум два преимущества векторного представления документов на основе bag-of-words/TF-IDF. Такие представления простые, быстрые и хорошо работают как базовый метод для классификации и поиска документов. TF-IDF дополнительно понижает вес слишком частых слов и повышает вес более информативных терминов.

4. TF-IDF: слово встречается 3 раза в документе из 30 термов, коллекция 1000 документов, слово есть в 10 документах.

$$TF = \frac{3}{30} = 0.1$$

$$IDF = \log_{10} \frac{1000}{10} = \log_{10} 100 = 2$$

$$TF-IDF = 0.1 \cdot 2 = 0.2$$

5. Однослойный двунаправленный nn.LSTM, batch_first=True, hidden_size=N, вход (B,T,D). Форма output:

$$(B, T, 2H)$$

Потому что для каждого токена объединяются два скрытых состояния: прямое направление и обратное направление.

6. Формула обновления (c_t) в LSTM. Что будет, если ($f_t=0$)?

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

где (f_t) — forget gate, (i_t) — input gate, $\tilde{c}_t$ — кандидат новой памяти. Если ($f_t=0$), старая память (c_{t-1}) полностью забывается.

7. Когда симметричная квантизация уступает асимметричной? Когда значения сильно смещены относительно нуля. Например, если активации лежат в диапазоне $([2;10])$, симметричная шкала тратит уровни на отрицательные значения, которых нет, а асимметричная лучше использует весь диапазон.

8. Минимум два преимущества адаптеров перед полным fine-tuning. Адаптеры обучают только небольшие дополнительные слои, поэтому требуют меньше памяти и вычислений. Также они не меняют основную модель, уменьшают риск катастрофического забывания и позволяют быстро переключаться между задачами.

9. Слой (W) размерности 1024×1024 . Сколько параметров добавит LoRA при $(r=8)$? LoRA добавляет две матрицы:

$$A \in \mathbb{R}^{1024 \times 8}, \quad B \in \mathbb{R}^{8 \times 1024}$$

$$1024 \cdot 8 + 8 \cdot 1024 = 8192 + 8192 = 16384$$

Итого: **16 384 обучаемых параметра.**

10. Минимум два преимущества vLLM перед базовым Hugging Face inference. vLLM эффективнее работает с KV-cache благодаря PagedAttention. Также он лучше поддерживает батчинг запросов, повышает throughput и лучше использует GPU при одновременной генерации для многих пользователей.

0.48 3. Практико-ориентированное задание

```
import torch.nn as nn
import torch.nn.functional as F

class AdditiveAttention(nn.Module):
    def __init__(self, enc_dim, dec_dim, attn_dim):
        super().__init__()
        self.W_h = nn.Linear(enc_dim, attn_dim)
        self.W_s = nn.Linear(dec_dim, attn_dim)
        self.v = nn.Linear(attn_dim, 1)

    def forward(self, encoder_states, decoder_state):
        # encoder_states: (B, T, H_enc)
        # decoder_state: (B, H_dec)

        # 1. Оценка соответствия decoder state каждому encoder state
        dec = self.W_s(decoder_state).unsqueeze(1)  # (B, 1, A)
        enc = self.W_h(encoder_states)              # (B, T, A)

        scores = self.v(torch.tanh(enc + dec)).squeeze(-1)  # (B, T)

        # 2. Преобразование оценок в веса внимания
        weights = F.softmax(scores, dim=1)  # (B, T)

        # 3. Контекстный вектор как взвешенная сумма hidden states энкодера
        context = torch.bmm(
            weights.unsqueeze(1),  # (B, 1, T)
            encoder_states         # (B, T, H_enc)
        ).squeeze(1)              # (B, H_enc)

        return context, weights

# Пример использования
```

```

B, T, H_enc, H_dec = 2, 5, 64, 64

encoder_states = torch.randn(B, T, H_enc)
decoder_state = torch.randn(B, H_dec)

attention = AdditiveAttention(enc_dim=H_enc, dec_dim=H_dec, attn_dim=32)

context, weights = attention(encoder_states, decoder_state)

print("context shape:", context.shape) # (B, H_enc)
print("weights shape:", weights.shape) # (B, T)
print("attention weights:", weights)

```

0.48.1 Формула аддитивного внимания

$$e_{t,i} = v_a^T \tanh(W_s s_t + W_h h_i)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}$$

$$c_t = \sum_i \alpha_{t,i} h_i$$

где (s_t) — текущее состояние декодера, (h_i) — скрытое состояние энкодера для (i) -го слова, $(e_{t,i})$ — оценка соответствия, $\alpha_{t,i}$ — вес внимания, (c_t) — контекстный вектор.

0.48.2 Какую проблему решает attention

В классическом encoder-decoder без attention всё входное предложение сжимается в один вектор последнего состояния энкодера. Это создаёт «бутылочное горлышко», особенно при длинных предложениях. Attention решает проблему тем, что декодер на каждом шаге смотрит не только на один итоговый вектор, а на все скрытые состояния энкодера и выбирает наиболее важные части входа.